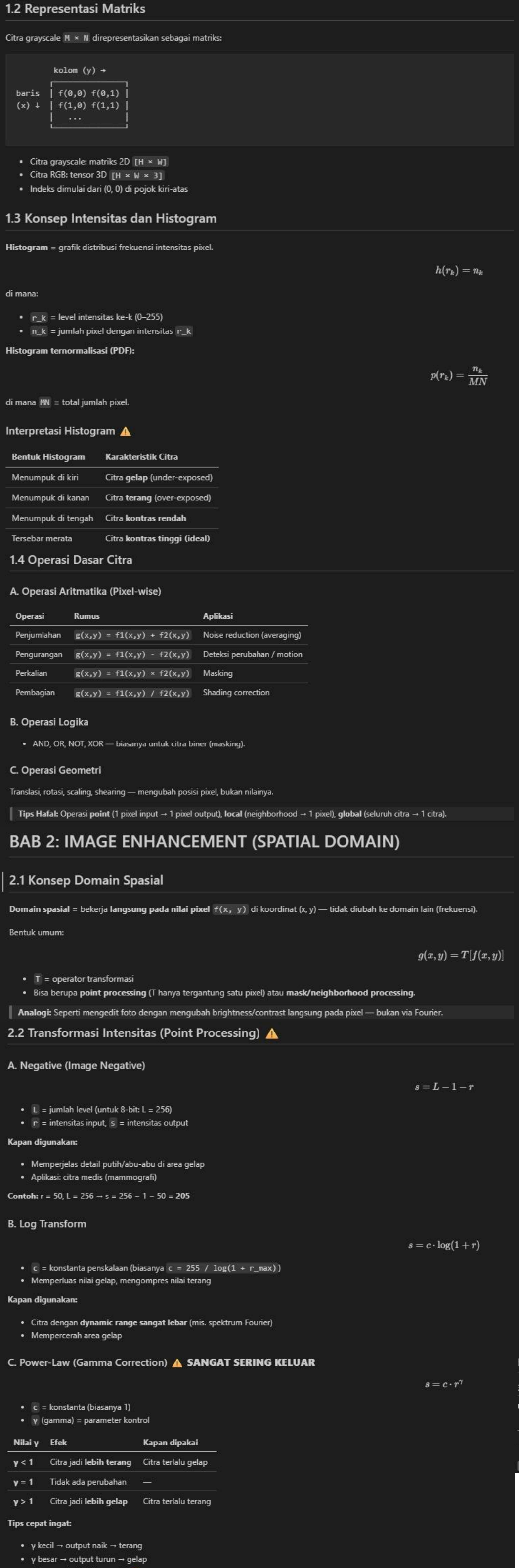


1.2 Representasi Matriks

Citra grayscale `M × N` direpresentasikan sebagai matriks:

```
        kolom (y) →
       ┌──────────────┐
baris  │ f(0,0) f(0,1) │
(x) ↓  │ f(1,0) f(1,1) │
       │   ...         │
       └──────────────┘
```

- Citra grayscale: matriks 2D `[H × W]`
- Citra RGB: tensor 3D `[H × W × 3]`
- Indeks dimulai dari (0, 0) di pojok kiri-atas

1.3 Konsep Intensitas dan Histogram

Histogram = grafik distribusi frekuensi intensitas pixel.

$$h(r_k) = n_k$$

di mana:

- `r_k` = level intensitas ke-k (0–255)
- `n_k` = jumlah pixel dengan intensitas `r_k`

Histogram ternormalisasi (PDF):

$$p(r_k) = \frac{n_k}{MN}$$

di mana `MN` = total jumlah pixel.

Interpretasi Histogram ⚠

Bentuk Histogram	Karakteristik Citra
Menumpuk di kiri	Citra **gelap** (under-exposed)
Menumpuk di kanan	Citra **terang** (over-exposed)
Menumpuk di tengah	Citra **kontras rendah**
Tersebar merata	Citra **kontras tinggi (ideal)**

1.4 Operasi Dasar Citra

A. Operasi Aritmatika (Pixel-wise)

Operasi	Rumus	Aplikasi
Penjumlahan	`g(x,y) = f1(x,y) + f2(x,y)`	Noise reduction (averaging)
Pengurangan	`g(x,y) = f1(x,y) - f2(x,y)`	Deteksi perubahan / motion
Perkalian	`g(x,y) = f1(x,y) × f2(x,y)`	Masking
Pembagian	`g(x,y) = f1(x,y) / f2(x,y)`	Shading correction

B. Operasi Logika

- AND, OR, NOT, XOR — biasanya untuk citra biner (masking).

C. Operasi Geometri

Translasi, rotasi, scaling, shearing — mengubah posisi pixel, bukan nilainya.

> **Tips Hafal:** Operasi **point** (1 pixel input → 1 pixel output), **local** (neighborhood → 1 pixel), **global** (seluruh citra → 1 citra).

BAB 2: IMAGE ENHANCEMENT (SPATIAL DOMAIN)

2.1 Konsep Domain Spasial

Domain spasial = bekerja **langsung pada nilai pixel** `f(x, y)` di koordinat (x, y) — tidak diubah ke domain lain (frekuensi).

Bentuk umum:

$$g(x,y) = T[f(x,y)]$$

- `T` = operator transformasi
- Bisa berupa **point processing** (T hanya tergantung satu pixel) atau **mask/neighborhood processing**.

> **Analogi:** Seperti mengedit foto dengan mengubah brightness/contrast langsung pada pixel — bukan via Fourier.

2.2 Transformasi Intensitas (Point Processing) ⚠

A. Negative (Image Negative)

$$s = L - 1 - r$$

- `L` = jumlah level (untuk 8-bit: L = 256)
- `r` = intensitas input, `s` = intensitas output

Kapan digunakan:

- Memperjelas detail putih/abu-abu di area gelap
- Aplikasi: citra medis (mammografi)

Contoh: r = 50, L = 256 → s = 256 − 1 − 50 = **205**

B. Log Transform

$$s = c \cdot \log(1 + r)$$

- `c` = konstanta penskalaan (biasanya `c = 255 / log(1 + r_max)`)
- Memperluas nilai gelap, mengompres nilai terang

Kapan digunakan:

- Citra dengan **dynamic range sangat lebar** (mis. spektrum Fourier)
- Mempercerah area gelap

C. Power-Law (Gamma Correction) ⚠ **SANGAT SERING KELUAR**

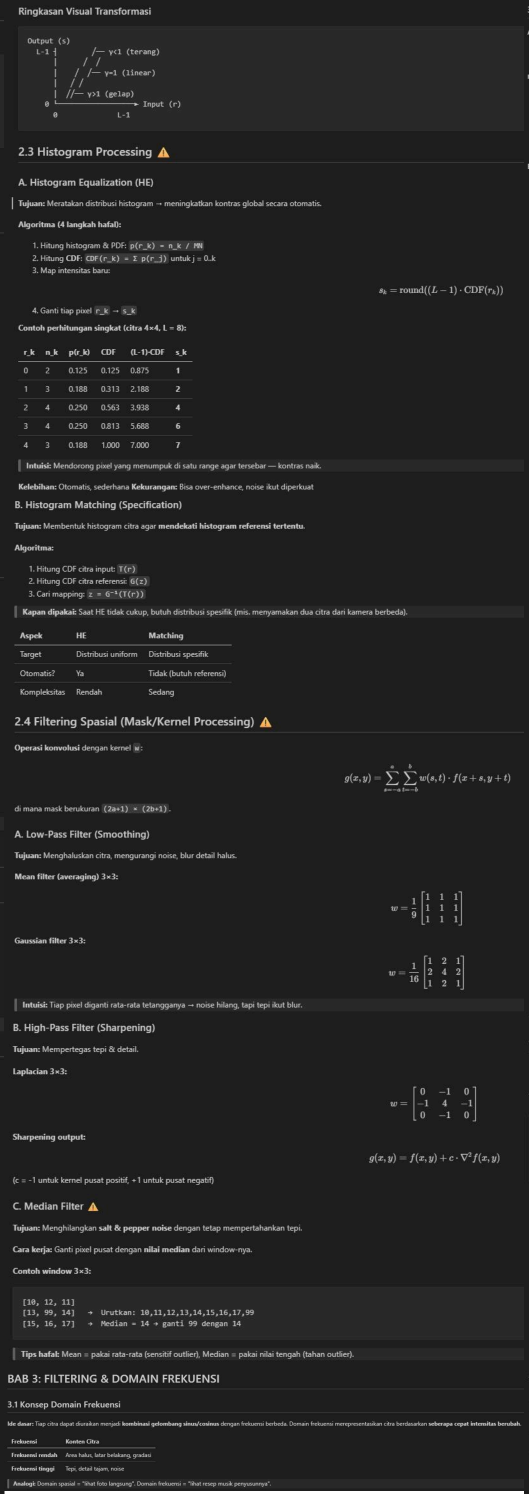
$$s = c \cdot r^{\gamma}$$

- `c` = konstanta (biasanya 1)
- `γ` (gamma) = parameter kontrol

Nilai γ	Efek	Kapan dipakai
γ < 1	Citra jadi **lebih terang**	Citra terlalu gelap
γ = 1	Tidak ada perubahan	—
γ > 1	Citra jadi **lebih gelap**	Citra terlalu terang

Tips cepat ingat:

- γ kecil → output naik → terang
- γ besar → output turun → gelap



Ringkasan Visual Transformasi

```
Output (s)
  L-1 ┤        /— γ<1 (terang)
      │      / /
      │    /  /— γ=1 (linear)
      │   / /
      │  //— γ>1 (gelap)
    0 └──────────────→ Input (r)
      0              L-1
```

2.3 Histogram Processing ⚠

A. Histogram Equalization (HE)

> **Tujuan:** Meratakan distribusi histogram → meningkatkan kontras global secara otomatis.

Algoritma (4 langkah hafal):

1. Hitung histogram & PDF: `p(r_k) = n_k / MN`
2. Hitung **CDF**: `CDF(r_k) = Σ p(r_j)` untuk j = 0..k
3. Map intensitas baru:

$$s_k = \text{round}((L-1) \cdot \text{CDF}(r_k))$$

4. Ganti tiap pixel `r_k` → `s_k`

Contoh perhitungan singkat (citra 4×4, L = 8):

r_k	n_k	p(r_k)	CDF	(L-1)·CDF	s_k
0	2	0.125	0.125	0.875	**1**
1	3	0.188	0.313	2.188	**2**
2	4	0.250	0.563	3.938	**4**
3	4	0.250	0.813	5.688	**6**
4	3	0.188	1.000	7.000	**7**

> **Intuisi:** Mendorong pixel yang menumpuk di satu range agar tersebar — kontras naik.

Kelebihan: Otomatis, sederhana **Kekurangan:** Bisa over-enhance, noise ikut diperkuat

B. Histogram Matching (Specification)

Tujuan: Membentuk histogram citra agar **mendekati histogram referensi tertentu.**

Algoritma:

1. Hitung CDF citra input: `T(r)`
2. Hitung CDF citra referensi: `G(z)`
3. Cari mapping: `z = G⁻¹(T(r))`

> **Kapan dipakai:** Saat HE tidak cukup, butuh distribusi spesifik (mis. menyamakan dua citra dari kamera berbeda).

Aspek	HE	Matching
Target	Distribusi uniform	Distribusi spesifik
Otomatis?	Ya	Tidak (butuh referensi)
Kompleksitas	Rendah	Sedang

2.4 Filtering Spasial (Mask/Kernel Processing) ⚠

Operasi konvolusi dengan kernel `w`:

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) \cdot f(x+s, y+t)$$

di mana mask berukuran `(2a+1) × (2b+1)`.

A. Low-Pass Filter (Smoothing)

Tujuan: Menghaluskan citra, mengurangi noise, blur detail halus.

Mean filter (averaging) 3×3:

$$w = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Gaussian filter 3×3:

$$w = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

> **Intuisi:** Tiap pixel diganti rata-rata tetangganya → noise hilang, tapi tepi ikut blur.

B. High-Pass Filter (Sharpening)

Tujuan: Mempertegas tepi & detail.

Laplacian 3×3:

$$w = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpening output:

$$g(x,y) = f(x,y) + c \cdot \nabla^2 f(x,y)$$

(c = -1 untuk kernel pusat positif, +1 untuk pusat negatif)

C. Median Filter ⚠

Tujuan: Menghilangkan **salt & pepper noise** dengan tetap mempertahankan tepi.

Cara kerja: Ganti pixel pusat dengan **nilai median** dari window-nya.

Contoh window 3×3:

```
[10, 12, 11]
[13, 99, 14]  →  Urutkan: 10,11,12,13,14,15,16,17,99
[15, 16, 17]  →  Median = 14 → ganti 99 dengan 14
```

> **Tips hafal:** Mean = pakai rata-rata (sensitif outlier), Median = pakai nilai tengah (tahan outlier).

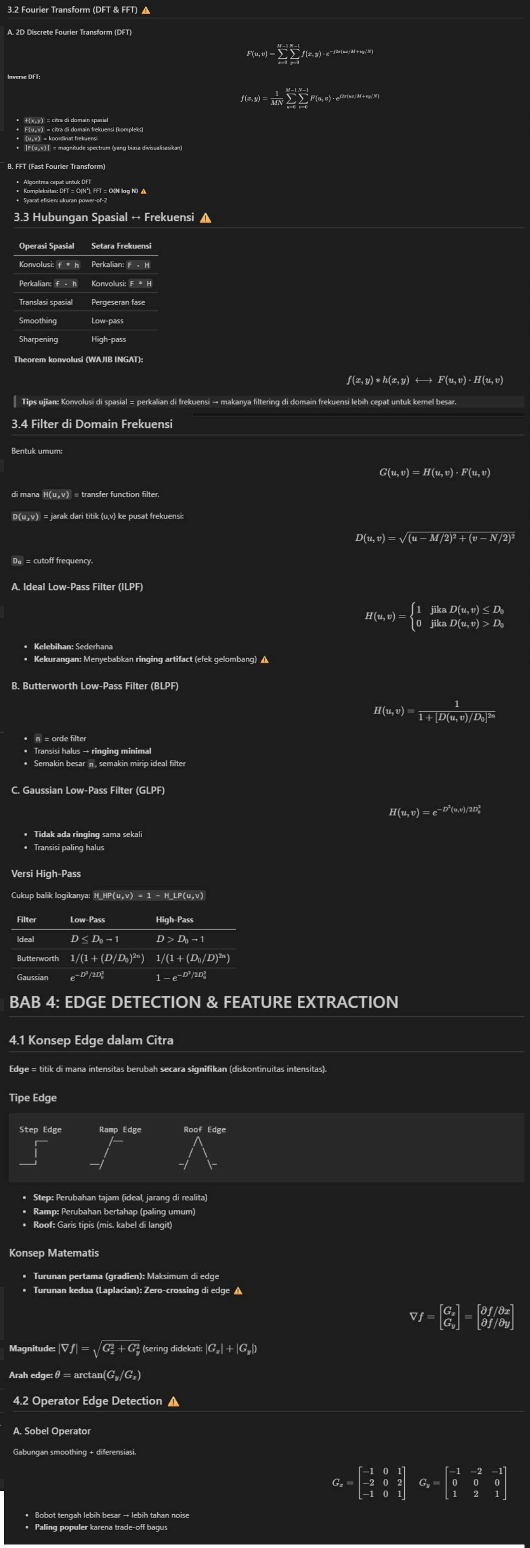
BAB 3: FILTERING & DOMAIN FREKUENSI

3.1 Konsep Domain Frekuensi

Ide dasar: Tiap citra dapat diuraikan menjadi **kombinasi gelombang sinus/cosinus** dengan frekuensi berbeda. Domain frekuensi merepresentasikan citra berdasarkan **seberapa cepat intensitas berubah.**

Frekuensi	Konten Citra
Frekuensi rendah	Area halus, latar belakang, gradasi
Frekuensi tinggi	Tepi, detail tajam, noise

> **Analogi:** Domain spasial = "lihat foto langsung". Domain frekuensi = "lihat resep musik penyusunnya".



3.2 Fourier Transform (DFT & FFT) ⚠

A. 2D Discrete Fourier Transform (DFT)

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cdot e^{-j2\pi(ux/M + vy/N)}$$

Inverse DFT:

$$f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) \cdot e^{j2\pi(ux/M + vy/N)}$$

- `f(x,y)` = citra di domain spasial
- `F(u,v)` = citra di domain frekuensi (kompleks)
- `(u,v)` = koordinat frekuensi
- `|F(u,v)|` = magnitude spectrum (yang biasa divisualisasikan)

B. FFT (Fast Fourier Transform)

- Algoritma cepat untuk DFT
- Kompleksitas: DFT = O(N^2), FFT = **O(N log N)** ⚠
- Syarat efisien: ukuran power-of-2

3.3 Hubungan Spasial ↔ Frekuensi ⚠

Operasi Spasial	Setara Frekuensi
Konvolusi: `f * h`	Perkalian: `F · H`
Perkalian: `f · h`	Konvolusi: `F * H`
Translasi spasial	Pergeseran fase
Smoothing	Low-pass
Sharpening	High-pass

Theorem konvolusi (WAJIB INGAT):

$$f(x,y) * h(x,y) \longleftrightarrow F(u,v) \cdot H(u,v)$$

> **Tips ujian:** Konvolusi di spasial = perkalian di frekuensi → makanya filtering di domain frekuensi lebih cepat untuk kernel besar.

3.4 Filter di Domain Frekuensi

Bentuk umum:

$$G(u,v) = H(u,v) \cdot F(u,v)$$

di mana `H(u,v)` = transfer function filter.

`D(u,v)` = jarak dari titik (u,v) ke pusat frekuensi:

$$D(u,v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

`D₀` = cutoff frequency.

A. Ideal Low-Pass Filter (ILPF)

$$H(u,v) = \begin{cases} 1 & \text{jika } D(u,v) \le D_0 \\ 0 & \text{jika } D(u,v) > D_0 \end{cases}$$

- **Kelebihan:** Sederhana
- **Kekurangan:** Menyebabkan **ringing artifact** (efek gelombang) ⚠

B. Butterworth Low-Pass Filter (BLPF)

$$H(u,v) = \frac{1}{1 + [D(u,v)/D_0]^{2n}}$$

- `n` = orde filter
- Transisi halus → **ringing minimal**
- Semakin besar `n`, semakin mirip ideal filter

C. Gaussian Low-Pass Filter (GLPF)

$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$

- **Tidak ada ringing** sama sekali
- Transisi paling halus

Versi High-Pass

Cukup balik logikanya: `H_HP(u,v) = 1 - H_LP(u,v)`

Filter	Low-Pass	High-Pass
Ideal	$D \le D_0$ → 1	$D > D_0$ → 1
Butterworth	$1/(1 + (D/D_0)^{2n})$	$1/(1 + (D_0/D)^{2n})$
Gaussian	$e^{-D^2/2D_0^2}$	$1 - e^{-D^2/2D_0^2}$

BAB 4: EDGE DETECTION & FEATURE EXTRACTION

4.1 Konsep Edge dalam Citra

Edge = titik di mana intensitas berubah **secara signifikan** (diskontinuitas intensitas).

Tipe Edge

```
Step Edge      Ramp Edge      Roof Edge
   ┌──            /──             /\
   │             /               /  \
───┘          ──/              -/    \-
```

- **Step:** Perubahan tajam (ideal, jarang di realita)
- **Ramp:** Perubahan bertahap (paling umum)
- **Roof:** Garis tipis (mis. kabel di langit)

Konsep Matematis

- **Turunan pertama (gradien):** Maksimum di edge
- **Turunan kedua (Laplacian): Zero-crossing** di edge ⚠

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}$$

Magnitude: $|\nabla f| = \sqrt{G_x^2 + G_y^2}$ (sering didekati: $|G_x| + |G_y|$)

Arah edge: $\theta = \arctan(G_y/G_x)$

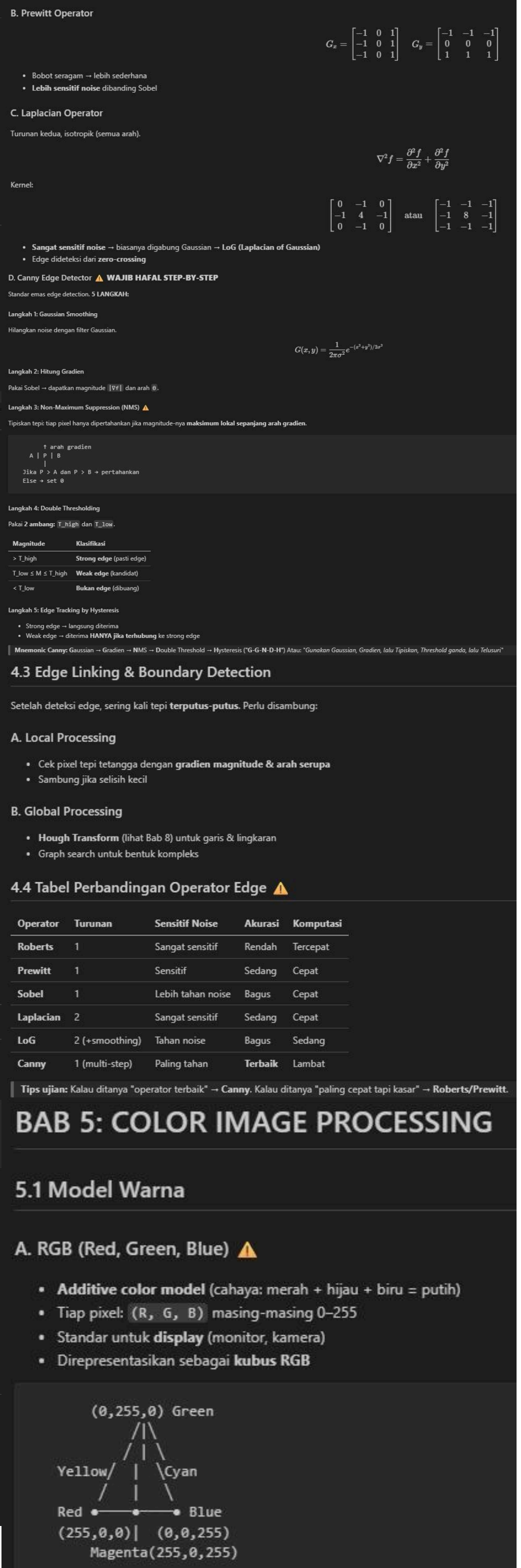
4.2 Operator Edge Detection ⚠

A. Sobel Operator

Gabungan smoothing + diferensiasi.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- Bobot tengah lebih besar → lebih tahan noise
- **Paling populer** karena trade-off bagus



B. Prewitt Operator

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- Bobot seragam → lebih sederhana
- **Lebih sensitif noise** dibanding Sobel

C. Laplacian Operator

Turunan kedua, isotropik (semua arah).

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \text{atau} \quad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- **Sangat sensitif noise** → biasanya digabung Gaussian → **LoG (Laplacian of Gaussian)**
- Edge dideteksi dari **zero-crossing**

D. Canny Edge Detector ⚠ **WAJIB HAFAL STEP-BY-STEP**

Standar emas edge detection. **5 LANGKAH:**

Langkah 1: Gaussian Smoothing

Hilangkan noise dengan filter Gaussian.

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

Langkah 2: Hitung Gradien

Pakai Sobel → dapatkan magnitude `|∇f|` dan arah `θ`.

Langkah 3: Non-Maximum Suppression (NMS) ⚠

Tipiskan tepi: tiap pixel hanya dipertahankan jika magnitude-nya **maksimum lokal sepanjang arah gradien.**

```
      ↑ arah gradien
  A | P | B
      |
Jika P > A dan P > B → pertahankan
Else → set 0
```

Langkah 4: Double Thresholding

Pakai **2 ambang:** `T_high` dan `T_low`.

Magnitude	Klasifikasi
> T_high	**Strong edge** (pasti edge)
T_low ≤ M ≤ T_high	**Weak edge** (kandidat)
< T_low	**Bukan edge** (dibuang)

Langkah 5: Edge Tracking by Hysteresis

- Strong edge → langsung diterima
- Weak edge → diterima **HANYA jika terhubung** ke strong edge

> **Mnemonic Canny:** **G**aussian → Gradien → **N**MS → **D**ouble Threshold → **H**ysteresis ("**G-G-N-D-H**") Atau: "*Gunakan Gaussian, Gradien, lalu Tipiskan, Threshold ganda, lalu Telusuri*"

4.3 Edge Linking & Boundary Detection

Setelah deteksi edge, sering kali tepi **terputus-putus.** Perlu disambung:

A. Local Processing

- Cek pixel tepi tetangga dengan **gradien magnitude & arah serupa**
- Sambung jika selisih kecil

B. Global Processing

- **Hough Transform** (lihat Bab 8) untuk garis & lingkaran
- Graph search untuk bentuk kompleks

4.4 Tabel Perbandingan Operator Edge ⚠

Operator	Turunan	Sensitif Noise	Akurasi	Komputasi
Roberts	1	Sangat sensitif	Rendah	Tercepat
Prewitt	1	Sensitif	Sedang	Cepat
Sobel	1	Lebih tahan noise	Bagus	Cepat
Laplacian	2	Sangat sensitif	Sedang	Cepat
LoG	2 (+smoothing)	Tahan noise	Bagus	Sedang
Canny	1 (multi-step)	Paling tahan	**Terbaik**	Lambat

> **Tips ujian:** Kalau ditanya "operator terbaik" → **Canny.** Kalau ditanya "paling cepat tapi kasar" → **Roberts/Prewitt.**

BAB 5: COLOR IMAGE PROCESSING

5.1 Model Warna

A. RGB (Red, Green, Blue) ⚠

- **Additive color model** (cahaya: merah + hijau + biru = putih)
- Tiap pixel: `(R, G, B)` masing-masing 0–255
- Standar untuk **display** (monitor, kamera)
- Direpresentasikan sebagai **kubus RGB**

```
        (0,255,0) Green
            /|\
           / | \
    Yellow/  |  \Cyan
         /   |   \
    Red ●────●────● Blue
    (255,0,0)|  (0,0,255)
        Magenta(255,0,255)
```